DSA01 – Assignment 2 (CO2)

ASSIGNMENT 2 – CO2

C02: Design C++ programs by applying functions and object-oriented concepts – function prototyping, call by reference, return by reference, inline functions, default arguments, function overloading, specifying a class, member functions, access specifiers, arrays within a class, static data members and member functions, arrays of objects and friend functions.

Student Name	G.RAJU	**Register Number**	192411065
Department / Section	CSE	**Academic Year / Semester**	
Faculty Name	Dr.Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	12-11-2026	**Date of Evaluation**	
GitHub Repository	DSAO115-192411065	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

OVERALL REMARKS

G.RAJU		
Student Signature	**Faculty Signature**	**Program Director**

Q1 – Call by Value, Reference and Pointer

1. Aim

To implement and compare call by value, call by reference and call by pointer in C++ using a hostel fee concession system.

2. Concept explanation

A hostel applies three types of concessions:

Concession	Percentage
Merit concession	10%
Sports concession	15%
Single-parent concession	25%

There are three ways to pass the fee to a function.

- Call by value: A copy of the fee is passed. Original fee remains unchanged.
- Call by reference: The function accesses the original variable through a reference.
- Call by pointer: The address of the original variable is passed and modified using *.

3. C++ program – Three implementations

File name: Q1_Concession.cpp

```
#include <iostream>

using namespace std;


// Call by value
```

```
void applyByValue(long long fee, int percent)
{
    fee -= fee * percent / 100;
}

// Call by reference
void applyByReference(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

// Call by pointer
void applyByPointer(long long* fee, int percent)
{
    *fee -= (*fee) * percent / 100;
}

int main()
{
    long long fee1 = 45000;
    long long fee2 = 45000;
    long long fee3 = 45000;
```

```
    applyByValue(fee1, 10);
    applyByReference(fee2, 15);
    applyByPointer(&fee3, 25);

    cout << "Call by Value: " << fee1 << endl;
    cout << "Call by Reference: " << fee2 << endl;
    cout << "Call by Pointer: " << fee3 << endl;

    return 0;
}
```

4. Expected output

Call by Value: 45000

Call by Reference: 38250

Call by Pointer: 33750

5. Predict the output – Given program

Prediction before execution:

1 12 13

1 100

16 13 6

Explanation:

Initially:

int a = 1, b = 2, c = 3;

After calling change(a, b, &c):

- a remains 1 because it is passed by value.
- b becomes 12 because it is passed by reference.
- c becomes 13 because it is passed by pointer.

After calling:

bigger(a, c) = 100;

The function returns a reference to the bigger variable. Since c is bigger, c becomes 100.

For show():

Function call	Default arguments used	Output
show(1)	y=5, z=10	16
show(1,2)	z=10	13
show(1,2,3)	None	6

6. Complete the Code – TODO 1

```
#include <iostream>

using namespace std;


void applyByValue(long long fee, int percent)

{

    fee -= fee * percent / 100;

}
```

```
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

    cout << "after call by value : "
         << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: "
         << hostelFee << '\n';

    return 0;
}
```

Expected output:

after call by value : 45000

after call by reference: 40500

7. Inquiry and design decision

Why does call by value leave the fee unchanged?

Call by value creates a separate local copy of the original fee. Any changes made inside the function affect only that copy.

Where are the parameters stored?

- Value: Separate local parameter.
- Reference: An alias to the caller's variable.
- Pointer: A pointer variable containing the address of the caller's variable.

When should const reference be used?

A const reference is useful when passing a large object to a function that only needs to read its data. It avoids copying and prevents accidental modification.

Design decision:

For multiple concessions, I would calculate the final fee and return it to the caller. This makes the function easier to test and avoids unexpected changes to the original fee. If the application requires updating the student's stored fee directly, call by reference is also appropriate.

8. Alternative concept and complexity analysis

Alternative implementation:

```
long long applyConcession(long long fee, int percent)

{

   return fee - fee * percent / 100;

}
```

// Calling:

hostelFee = applyConcession(hostelFee, 10);

Method	Time complexity	Parameter copying
Call by value	O(1)	Copies long long
Call by reference	O(1)	No object copy
Call by pointer	O(1)	Copies address
Const reference	O(1)	No object copy
Return by value	O(1)	Returns the result

For a large Student object, passing by value copies the complete object, whereas reference and const reference avoid that copy.

9. Conclusion

Thus, call by value, reference and pointer were implemented successfully. Reference and pointer modify the original hostel fee, whereas call by value modifies only a local copy.

Q2 – Return by Reference and Its Dangers

4 Marks

1. Aim

To understand return by reference and demonstrate how a function can modify an array element directly through a returned reference.

2. Concept explanation

Return by reference means that a function returns an alias to an existing variable instead of returning a separate copy.

It allows the caller to modify the original variable.

3. C++ program

File name: Q2_ReturnReference.cpp

```
#include <iostream>

using namespace std;


// Returns reference to the highest mark

int& highestMark(int marks[], int size)

{

   int maxIndex = 0;


   for (int i = 1; i < size; i++)

   {

      if (marks[i] > marks[maxIndex])

      {

         maxIndex = i;

      }

   }
```

```
    return marks[maxIndex];
}

// Returns a reference to a local variable
int& dangerousFunction()
{
    int x = 50;
    return x;  // Dangling reference - undefined behaviour
}

int main()
{
    int marks[5] = {75, 85, 92, 80, 88};

    cout << "Highest mark: "
         << highestMark(marks, 5) << endl;

    highestMark(marks, 5) += 2;

    cout << "After moderation: "
         << highestMark(marks, 5) << endl;

    highestMark(marks, 5) = 100;
```

```
    cout << "After updating: "
         << highestMark(marks, 5) << endl;

    return 0;
}
```

Important: The dangerous function is included to demonstrate the error, but it is deliberately not called. Calling it and accessing its returned reference would cause undefined behaviour.

4. Expected output

Highest mark: 92

After moderation: 94

After updating: 100

5. Explanation

Initially, the marks are:

75 85 92 80 88

The highest mark is 92.

When the statement executes:

highestMark(marks, 5) += 2;

The original array element changes from 92 to 94.

When:

highestMark(marks, 5) = 100;

The original array element changes to 100.

6. Compiler warning and dangling reference

Consider:

```
int& dangerousFunction()
{
    int x = 50;
    return x;
}
```

The variable x is a local variable. Its lifetime ends when the function returns.

The returned reference therefore refers to an object that no longer exists.

A compiler such as GCC may produce a warning similar to:

warning: reference to local variable 'x' returned

The exact warning text can vary by compiler version.

7. Difference between reference, const reference and copy

Method	Description	Modification
int&	Returns original variable	Allowed
const int&	Returns read-only reference	Not allowed
int	Returns separate copy	Original unchanged

8. Changed condition – std::vector

When an array is replaced by a std::vector, adding more elements may cause the vector to reallocate its memory.

If reallocation occurs, previously obtained references and pointers to its elements may become invalid.

Therefore, references should not be kept and reused after operations that may cause reallocation.

9. Alternative concept

Returning an index:

```
int highestIndex(int marks[], int size)

{

   int index = 0;



   for (int i = 1; i < size; i++)

   {

      if (marks[i] > marks[index])

         index = i;

   }



   return index;

}
```

Calling:

```
int index = highestIndex(marks, 5);

marks[index] += 2;
```

Returning a pointer:

```
int* highestPointer(int marks[], int size)

{
```

```
    int* maximum = &marks[0];


    for (int i = 1; i < size; i++)
    {
        if (marks[i] > *maximum)
            maximum = &marks[i];
    }


    return maximum;
}
```

Calling:

```
*highestPointer(marks, 5) += 2;
```

10. Complexity analysis

Approach	Time	Extra space
Return reference	O(n)	O(1)
Return index	O(n)	O(1)
Return pointer	O(n)	O(1)

All three approaches require O(n) time to find the highest mark.

Design decision: Returning an index is a good choice when working with a vector because the index can be used again after reallocation, provided the logical element ordering remains unchanged and the index is still valid.

11. Conclusion

Return by reference allows direct modification of an existing variable. However, returning a reference to a local variable is unsafe because its lifetime ends when the function returns.

Q3 – Default Arguments and Function Overloading

6 Marks

1. Aim

To implement a courier charge calculator using default arguments and function overloading without causing ambiguity.

2. Concept explanation

Function overloading allows multiple functions to have the same name but different parameter lists.

Default arguments provide predefined values when the caller does not supply optional arguments.

The original design is ambiguous because:

charge(double weight);

charge(double weight, bool express);

charge(double weight,

bool express = false,

bool insured = false);

A call such as charge(2) can match both the first and third functions.

The compiler cannot choose a unique best function.

3. Redesigned C++ program

File name: Q3_Courier.cpp

```
#include <iostream>
using namespace std;

struct Parcel
{
    double weight;
    bool express;
    bool insured;
};

// Weight in grams
void charge(int grams)
{
    cout << "Integer grams: "
         << grams << " g" << endl;
}

// Weight in kilograms
void charge(double kg,
            bool express = false,
            bool insured = false)
```

```
{
    double cost = kg * 50;

    if (express)
        cost += 30;

    if (insured)
        cost += 20;

    cout << "Weight in kg: "
         << cost << endl;
}

// Parcel object
void charge(const Parcel& p)
{
    double cost = p.weight * 50;

    if (p.express)
        cost += 30;

    if (p.insured)
        cost += 20;
```

```
    cout << "Parcel charge: "
         << cost << endl;
}

int main()
{
    Parcel p = {3.0, true, true};

    charge(2);
    charge(2.0f);
    charge(2.5);
    charge(2.5, true);
    charge(2.5, true, true);
    charge(p);

    return 0;
}
```

4. Predicted function selection

Call	Selected function	Reason
charge(2)	charge(int)	Exact match

charge(2.0f)	charge(double)	Float to double promotion
charge(2.5)	charge(double)	Exact match
charge(2.5,true)	charge(double,bool,bool)	Default insured argument
charge(2.5,true,true)	charge(double,bool,bool)	All arguments supplied
charge(p)	charge(const Parcel&)	Parcel object match

5. Expected output

Integer grams: 2 g

Weight in kg: 100

Weight in kg: 125

Weight in kg: 155

Weight in kg: 175

Parcel charge: 200

6. Explanation of overload resolution

The compiler generally prefers:

1. Exact match.
2. Promotion.
3. Standard conversion.

For example:

- Integer 2 exactly matches int.
- Float 2.0f is promoted to double.
- A double value exactly matches the double parameter.

Default arguments must be provided from the right side because arguments are supplied from left to right.

Correct:

void display(int a, int b = 10, int c = 20);

Incorrect:

void display(int a = 10, int b, int c = 20);

7. Alternative concept

A single Parcel structure can hold all optional information.

```
struct Parcel
{
    double weight;
    bool express;
    bool insured;
};
```

A single function can receive the Parcel object and calculate the charge.

This design makes adding new options easier because new fields can be added to the structure.

8. Complexity analysis

Design	Time	Extra space
Function overloading	O(1) calculation	Depends on parameters
Parcel structure	O(1) calculation	O(1) for fixed fields
Function template	O(1) for this calculation	Depends on argument type

Overload resolution happens during compilation, so it introduces no additional runtime search cost.

9. Conclusion

Function overloading and default arguments were implemented successfully. The redesigned functions handle different input types and avoid ambiguous calls.

Q4 – Specifying a Class with an Array Inside It

6 Marks

1. Aim

To design a Savings Account class using private data members, an array of transactions and public member functions.

2. Concept explanation

A SavingsAccount object stores:

- Account number.
- Current balance.
- Last 10 transactions.
- Number of transactions stored.

The class uses private members to protect account information and public member functions to perform deposits, withdrawals and display mini statements.

3. C++ program

File name: Q4_SavingsAccount.cpp

```
#include <iostream>

#include <string>

using namespace std;
```

```
class SavingsAccount
{
private:
    int accountNumber;
    double balance;
    double transactions[10];
    int count;

public:
    SavingsAccount(int number, double initialBalance)
    {
        accountNumber = number;
        balance = initialBalance;
        count = 0;
    }

    void addTransaction(double amount)
    {
        // Discard the oldest transaction when full
        if (count == 10)
        {
            for (int i = 0; i < 9; i++)
```

```
            {
                transactions[i] = transactions[i + 1];
            }
            count = 9;
        }

        transactions[count] = amount;
        count++;
    }

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement() const;
};

void SavingsAccount::deposit(double amount)
{
    if (amount > 0)
    {
        balance += amount;
        addTransaction(amount);
        cout << "Deposit successful" << endl;
    }
```

```
    else
    {
        cout << "Invalid deposit" << endl;
    }
}

void SavingsAccount::withdraw(double amount)
{
    if (amount > 0 && amount <= balance)
    {
        balance -= amount;
        addTransaction(-amount);
        cout << "Withdrawal successful" << endl;
    }
    else
    {
        cout << "Insufficient balance or invalid amount" << endl;
    }
}

void SavingsAccount::miniStatement() const
{
    cout << "\nAccount Number: "
```

```
        << accountNumber << endl;

    cout << "Balance: " << balance << endl;

    cout << "Last Transactions:\n";

    for (int i = 0; i < count; i++)
    {
        cout << transactions[i] << endl;
    }
}

int main()
{
    SavingsAccount account(1001, 5000);

    account.deposit(1000);
    account.withdraw(500);
    account.deposit(2000);
    account.withdraw(300);

    account.miniStatement();
```

```
    return 0;
}
```

4. Expected output

Deposit successful

Withdrawal successful

Deposit successful

Withdrawal successful

Account Number: 1001

Balance: 7200

Last Transactions:

1000

-500

2000

-300

5. Inquiry

Why must the array be private?

The array must be private because transaction information should not be directly modified from outside the class. Private access protects the account data and allows modifications only through member functions.

What happens when the 11th transaction arrives?

When the array already contains 10 transactions, the oldest transaction is removed. The remaining nine transactions are shifted left, and the new transaction is inserted at the last position.

6. Alternative concept – Circular buffer

Instead of shifting all elements, a circular buffer maintains a starting index and overwrites the oldest transaction.

Pseudocode:

IF transaction count < 10

    INSERT transaction

    INCREASE count

ELSE

    REPLACE oldest transaction

    MOVE start index circularly

END IF

A std::deque can also be used:

```
deque<double> transactions;


transactions.push_back(amount);


if (transactions.size() > 10)
{
    transactions.pop_front();
}
```

7. Comparison

Method	Adding transaction	Space
Fixed array with shifting	O(k)	O(10)
Circular buffer	O(1)	O(10)
Deque	O(1) amortized at ends	O(10) retained elements

Here, k represents the number of stored transactions that must be shifted.

8. Design decision

If the bank increases the requirement from 10 to 50 transactions, the circular buffer design requires only changing its capacity from 10 to 50.

It avoids shifting existing elements and provides faster insertion.

9. Conclusion

The SavingsAccount class was implemented using private data members, an internal transaction array and public member functions. The circular buffer is preferable when frequent transaction updates are required.

Q5 – Static Members and Arrays of Objects

7 Marks

1. Aim

To implement static data members and static member functions in C++ using a college bus seat ticket allocation system.

2. Concept explanation

A static data member belongs to the entire class rather than an individual object.

All objects share the same static variable.

For the college bus:

- issued stores the number of allocated seats.
- CAPACITY stores the maximum seat capacity.
- seat stores the individual ticket's seat number.
- name stores the passenger's name.

The static counter is shared among all BusTicket objects.

3. C++ program

File name: Q5_BusTicket.cpp

```
#include <iostream>

#include <string>

using namespace std;


class BusTicket

{

    static int issued;

    static const int CAPACITY = 3;


    int seat;

    string name;


public:

    explicit BusTicket(const string& n)
```

```
    : seat(0), name(n)
{
    if (issued < CAPACITY)
    {
        issued++;
        seat = issued;
    }
}

static int count()
{
    return issued;
}

void print() const
{
    cout << name << ": ";

    if (seat != 0)
        cout << "Seat " << seat;
    else
        cout << "Waiting";
```

```
        cout << '\n';
    }
};

// Definition of static data member
int BusTicket::issued = 0;

int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
    {
        t.print();
    }

    cout << "Issued: "
         << BusTicket::count() << endl;
```

```
    return 0;
}
```

4. Expected output

Asha: Seat 1

Bala: Seat 2

Chitra: Seat 3

Dev: Waiting

Issued: 3

5. Explanation of TODO 1 and TODO 2

TODO 1:

```
if (issued < CAPACITY)
{
    issued++;
    seat = issued;
}
```

This checks whether a seat is available. If a seat is available, the counter increases and the ticket receives the next seat number.

Otherwise, the ticket remains in the waiting list with seat number 0.

TODO 2:

```
return issued;
```

This returns the number of seats issued so far.

6. Predict the output – Given Token program

Prediction before execution:

101 103 104 4

Explanation:

Initially:

int Token::next = 100;

The counter starts at 100.

Three Token objects are created in the array:

Object	ID
counter[0]	101
counter[1]	102
counter[2]	103
extra	104

The constructor executes four times.

The static function:

```
static int issued()
{
    return next - 100;
}
```

returns:

104 - 100 = 4

Therefore, the output is:

101 103 104 4

7. Creating tickets inside a function called twice

Example:

```
void createTickets()
{
    BusTicket tickets[2] = {
        BusTicket("Ravi"),
        BusTicket("Kiran")
    };

    for (const BusTicket& t : tickets)
        t.print();
}

int main()
{
    createTickets();
    createTickets();

    cout << "Total issued: "
         << BusTicket::count() << endl;
}
```

Expected output:

Ravi: Seat 1

Kiran: Seat 2

Ravi: Seat 3

Kiran: Waiting

Total issued: 3

The static variable is not reset when the function finishes. It continues to maintain its value across both function calls.

8. Why cannot a static member function use this?

A static member function belongs to the class and can be called without creating an object.

Therefore, it does not have an individual object address represented by this.

It can directly access static members but cannot directly access non-static members without an object.

9. Alternative concept

Using a global variable:

int issued = 0;

A global variable can maintain the ticket count, but it is not properly encapsulated inside the class.

Using a separate TicketOffice object:

class TicketOffice

{

    int issued = 0;

};

This approach is useful when multiple buses operate independently.

For example, Bus A and Bus B can have separate ticket counters.

10. Complexity analysis

Method	Time	Extra space
Static counter	O(1)	O(1)
Global counter	O(1)	O(1)
TicketOffice object	O(1)	O(1) per office

Design decision: A static counter is suitable for one bus because every ticket shares the same capacity. A separate TicketOffice object is better when multiple buses require independent seat allocations.

11. Conclusion

Static members and arrays of objects were implemented successfully. The program allocated three seats and placed the fourth passenger on the waiting list.

Q6 – Friend Functions and Inline Functions

8 Marks

1. Aim

To implement friend functions and inline functions using a Train Journey Planner that compares departure and arrival times.

2. Concept explanation

A friend function is not a member of a class, but it is allowed to access the private and protected members of that class.

It is useful when an operation needs to work equally on two objects.

An inline function is a function for which the compiler is permitted to substitute the function body at the call site when appropriate.

The inline keyword does not guarantee that the compiler will actually inline the function.

3. C++ program – Complete the Code

File name: Q6_Time.cpp

```
#include <iostream>

#include <cstdlib>

using namespace std;


class Time

{

   int h, m;


public:

   Time(int hh, int mm) : h(hh), m(mm) {}


   friend inline int minutesBetween(

      const Time& a, const Time& b);


   friend inline bool earlier(

      const Time& a, const Time& b);

};
```

```
inline int minutesBetween(
    const Time& a, const Time& b)
{
    int time1 = a.h * 60 + a.m;
    int time2 = b.h * 60 + b.m;

    return abs(time1 - time2);
}

inline bool earlier(
    const Time& a, const Time& b)
{
    int time1 = a.h * 60 + a.m;
    int time2 = b.h * 60 + b.m;

    return time1 < time2;
}

int main()
{
    Time departure(9, 45);
    Time arrival(13, 10);
```

```
cout << minutesBetween(departure, arrival)
    << ' '
    << boolalpha
    << earlier(arrival, departure)
    << '\n';

// Same time test
Time t1(12, 30);
Time t2(12, 30);

cout << minutesBetween(t1, t2)
    << ' '
    << earlier(t1, t2) << endl;

// Times on either side of noon
Time morning(11, 50);
Time afternoon(12, 10);

cout << minutesBetween(morning, afternoon)
    << ' '
    << earlier(morning, afternoon) << endl;
```

```
    return 0;
}
```

4. Expected output

205 false

0 false

20 true

5. Explanation of TODO 1

Convert each time into total minutes.

For departure:

9 × 60 + 45 = 585

For arrival:

13 × 60 + 10 = 790

Difference:

790 - 585 = 205 minutes

Therefore, minutesBetween() returns 205.

6. Explanation of TODO 2

The function checks whether the first time is earlier than the second time.

return time1 < time2;

In the given program:

earlier(arrival, departure)

13:10 is not earlier than 09:45.

Therefore, the result is false.

7. Why is a friend function needed?

A friend function can access the private members of the Time class.

It is useful when comparing two Time objects equally because neither object is treated as the main object.

A member function would generally be called using:

departure.isEarlier(arrival);

A friend function is called using:

earlier(departure, arrival);

8. Friend function versus member function

Friend function	Member function
Not a class member	Belongs to class
Called like a normal function	Called using object
Can access private data if declared friend	Can access its own object's private data
No this pointer	Has this pointer

9. What happens if private members are made public?

If the variables h and m are made public, any external code can directly modify them.

This reduces encapsulation and allows invalid values such as:

Time t(9, 30);

// Public data could be changed to 9 hours, 90 minutes.

Keeping the members private protects the internal representation of the Time class.

10. Alternative concept – Member functions

An alternative implementation can use member functions:

```
class Time
{
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   int minutesTo(const Time& other) const
   {
      int a = h * 60 + m;
      Int b = other.h * 60 + other.m;
      return abs(a - b);
   }
   bool isEarlier(const Time& other) const
   {
      return h * 60 + m <
            other.h * 60 + other.m;
   }
};
```

Calling:

```
departure.minutesTo(arrival);
departure.isEarlier(arrival);
```

Another alternative is to use public getter functions, but this exposes more of the class interface.

11. Complexity analysis

Approach	Time per call	Extra space
Friend function	O(1)	O(1)
Member function	O(1)	O(1)
Getter-based comparison	O(1)	O(1)

Inlining may reduce function-call overhead, especially for small functions, but the actual performance depends on compiler optimisation.

12. Inline performance measurement

The following benchmark can compare inline and non-inline versions.

File name: Q6_Benchmark.cpp

```
#include <iostream>

#include <chrono>

using namespace std;

using namespace chrono;


#ifdef USE_INLINE

inline int calculate(int x)

{

    return x * 2 + 1;

}
```

```
#else
int calculate(int x)
{
    return x * 2 + 1;
}
#endif
int main()
{
    volatile long long result = 0;
    const int N = 10000000;
    auto start = high_resolution_clock::now();
    for (int i = 0; i < N; i++)
    {
        result += calculate(i);

    auto stop = high_resolution_clock::now();
    auto duration =
        duration_cast<microseconds>(stop - start);
    cout << "Result: " << result << endl;
    cout << "Execution time: "
         << duration.count()
         << " microseconds" << endl;
    return 0;
```

}

Compile and measure four configurations:

g++ -std=c++17 -O0 Q6_Benchmark.cpp -o normal_O0

g++ -std=c++17 -O2 Q6_Benchmark.cpp -o normal_O2

g++ -std=c++17 -O0 -DUSE_INLINE Q6_Benchmark.cpp -o inline_O0

g++ -std=c++17 -O2 -DUSE_INLINE Q6_Benchmark.cpp -o inline_O2

Run each executable and record the measured execution time.

Configuration	**Measured time**
Normal -O0	Record your result
Normal -O2	Record your result
Inline -O0	Record your result
Inline -O2	Record your result

Do not fill this table with invented timings. Different computers and compilers produce different measurements.

13. Deployment decision

Friend functions are suitable when comparing two objects equally and when direct access to private members is required.

Member functions are preferable when the operation naturally belongs to one object.

For this Train Journey Planner, friend functions are justified because the comparison involves two Time objects equally.

14. Conclusion

Friend and inline functions were implemented successfully. The program calculated the difference between two times and compared their order while maintaining private data members.

Q7 – Diagnose, Redesign and Validate

13 Marks

1. Aim

To identify a limitation in an existing C++ program, redesign it using suitable data structures and evaluate the performance improvement.

2. Selected program

Q4 – Savings Account Mini-Statement

I selected Q4 because the original implementation uses an array and shifts all existing transactions whenever a new transaction arrives after the array becomes full.

This operation becomes inefficient when the number of stored transactions increases.

A. Diagnose the limitation – 2 Marks

In the original program, the transaction array stores only the last 10 transactions.

When the 11th transaction arrives, the program shifts the remaining nine elements to the left to make space for the new transaction.

For every additional transaction after the array becomes full, the shifting operation is repeated.

Original limitation:

- Unnecessary movement of existing elements.
- Increased insertion time.
- Less efficient when transaction capacity increases.
- The design does not provide constant-time insertion.

B. Define the design objective – 2 Marks

The objective is to redesign the transaction storage system using a circular buffer.

The revised system should:

1. Store the latest 50 transactions.
2. Remove the oldest transaction automatically.
3. Insert new transactions without shifting existing elements.
4. Maintain the correct transaction order.
5. Achieve O(1) insertion time.

C. Design the revised solution – 4 Marks

Proposed concept: Circular Buffer

A circular buffer uses a fixed-size array and a starting index.

When the buffer becomes full, the oldest transaction is replaced by the newest transaction.

Circular Buffer Wrap-Around: Elegant Mathematical Mastery

tech.hoomanely.com

Working procedure:

1. Initially, the start index is zero.
2. New transactions are inserted at the calculated circular position.
3. When the buffer becomes full, the oldest transaction is overwritten.
4. The start index moves to the next position.
5. The mini statement displays transactions from oldest to newest.

Revised C++ program

File name: Q7_CircularBuffer.cpp

```
#include <iostream>

using namespace std;

class SavingsAccount

{
```

```
private:
    static const int SIZE = 50;
    int accountNumber;
    double balance;
    double transactions[SIZE];
    int start;
    int count
public:
    SavingsAccount(int number, double initialBalance)
    {
        accountNumber = number;
        balance = initialBalance;
        start = 0;
        count = 0;
    }
    void addTransaction(double amount)
    {
        int position = (start + count) % SIZE;
        if (count == SIZE)
        {
            transactions[start] = amount;
            start = (start + 1) % SIZE;
        }
```

```
    else
    {
        transactions[position] = amount;
        count++;
    }
}
void deposit(double amount)
{
    if (amount > 0)
    {
        balance += amount;
        addTransaction(amount);
    }
}
void withdraw(double amount)
{
    if (amount > 0 && amount <= balance)
    {
        balance -= amount;
        addTransaction(-amount);
    }
}
void miniStatement() const
```

```
    {
        cout << "Account Number: "
             << accountNumber << endl;

        cout << "Balance: "
             << balance << endl;

        cout << "Last Transactions:\n";

        for (int i = 0; i < count; i++)
        {
            int position = (start + i) % SIZE;
            cout << transactions[position] << endl;
        }
    }
};

int main()
{
    SavingsAccount account(1001, 50000);
    for (int i = 1; i <= 55; i++)
    {
        account.deposit(100);
```

```
    }

    account.miniStatement();


    return 0;

}
```

Expected output

Account Number: 1001

Balance: 55500

Last Transactions:

100

100

100

...

100

There are exactly 50 transaction entries displayed.

The first five transactions are discarded because the system stores only the latest 50 transactions.

D. Predict before implementing – 1 Mark

My prediction is:

- The original array implementation requires shifting elements after reaching its capacity.
- The redesigned circular buffer will insert transactions without shifting.
- Both implementations will maintain the same latest transaction records.
- The redesigned version will require less insertion work when the buffer is full.

E. Implement and validate – 2 Marks

The following table compares the original and redesigned approaches.

Feature	Original design	Redesigned design
Data structure	Fixed array	Circular buffer
Capacity	10	50
New transaction	Shifting	Overwriting
Insertion time when full	O(k)	O(1)
Space complexity	O(10)	O(50)
Transaction order	Oldest to newest	Oldest to newest

Performance measurement

For a fair comparison, test both programs using the same number of transactions and the same capacity.

Use the following benchmark logic:

```
const int N = 1000000;

auto start = chrono::high_resolution_clock::now();

for (int i = 0; i < N; i++)

{

    account.addTransaction(100);

}

auto stop = chrono::high_resolution_clock::now();

auto duration =
```

```
    chrono::duration_cast<chrono::microseconds>(
        stop - start
    )
cout << "Execution time: "
     << duration.count() << " microseconds";
```

Include these headers:

```
#include <chrono>
```

For a proper comparison, create one version using shifting and another using the circular buffer, then measure both using identical input sizes.

Record your results:

Input transactions	**Original shifting**	**Circular buffer**
1,000	Measure	Measure
10,000	Measure	Measure
100,000	Measure	Measure
1,000,000	Measure	Measure

The results will depend on your computer and compiler. The theoretical improvement is the reduction from O(k) to O(1) insertion work when the buffer is full.

F. Defend the final design – 2 Marks

The redesigned circular buffer is more efficient because it avoids shifting existing transactions.

It is particularly useful for applications that frequently insert new transactions and maintain only a fixed number of recent records.

However, the original shifting implementation is simpler to understand and may be suitable for small transaction counts.

The circular buffer requires additional index management, but the performance benefit makes it a better choice for larger transaction histories.

3. Final complexity comparison

Operation	Original shifting	Circular buffer
Insert when not full	O(1)	O(1)
Insert when full	O(k)	O(1)
Display transactions	O(k)	O(k)
Search transaction	O(k)	O(k)
Storage	O(k)	O(k)

4. Conclusion

The original transaction storage system was redesigned using a circular buffer. The revised implementation eliminates unnecessary element shifting and provides constant-time insertion when the buffer is full.